

CLASE INTEGRADORA

Programación I

Criterios generales

- Resolver utilizando programación estructurada.
- No utilizar clases, diccionarios, conjuntos ni break.
- Se permite definir funciones auxiliares cuando resulte conveniente.
- Priorizar soluciones claras, legibles y justificables en una evaluación escrita.

EJERCICIO 1 – Organización de un festival de música

Matrices, listas, strings, funciones, tuplas y excepciones

La organización del festival SoundCity debe controlar la asignación de artistas a los distintos escenarios. El festival dispone de 4 escenarios y 5 franjas horarias.

Se cuenta con las siguientes listas:

```
artistas = ["Luz Norte", "Banda Cero", "Los Viajeros", "Marea", "Neon", "Sur"]
escenarios = ["Principal", "Norte", "Electro", "Acustico"]
horarios = ["16:00", "17:30", "19:00", "20:30", "22:00"]
```

Cada artista se identifica mediante su posición en la lista artistas, comenzando desde el código 1. Por ejemplo: 1 → Luz Norte, 2 → Banda Cero, 3 → Los Viajeros, etc.

La programación se almacena en una matriz de 4 filas por 5 columnas. Cada fila representa un escenario y cada columna una franja horaria. En cada posición se almacena 0 si la franja está libre o el código del artista asignado.

```
programacion = [
    [1, 0, 3, 0, 5],
    [0, 2, 0, 4, 0],
    [5, 0, 2, 0, 0],
    [0, 6, 0, 3, 0]
]
```

Se pide desarrollar las siguientes funciones:

a) `mostrar_programacion()`

Recibe la matriz, la lista de artistas, escenarios y horarios. Debe mostrar solamente las franjas ocupadas con el siguiente formato:

```
Principal - 16:00 - Luz Norte
Principal - 19:00 - Los Viajeros
```

b) `buscar_artista()`

Recibe nombre, artistas, programacion, escenarios y horarios. La búsqueda debe realizarse sin diferenciar mayúsculas de minúsculas y eliminando espacios al comienzo y al final.

La función debe devolver una lista de tuplas con todas las actuaciones encontradas. Cada tupla tendrá el formato (escenario, horario).

```
buscar_artista(" los viajeros ", artistas, programacion, escenarios, horarios)

Resultado esperado:
[("Principal", "19:00"), ("Acustico", "20:30")]
```

Si el artista no existe en la lista o no tiene actuaciones asignadas, deberá devolver una lista vacía.

c) `escenario_mas_ocupado()`

Recibe programacion y escenarios. Debe devolver una tupla (nombre_escenario, cantidad_actuaciones) correspondiente al escenario que tenga más franjas ocupadas. Si dos escenarios tienen la misma cantidad, se devuelve el primero.

d) leer_codigo_artista()

Solicita por teclado el código de un artista. Debe controlar mediante try/except que el dato pueda convertirse a entero. El código deberá encontrarse entre 1 y la cantidad de artistas. Mientras el valor no sea válido deberá volver a solicitarlo. La función devuelve el código válido.

EJERCICIO 2 – Sistema de equipaje de una aerolínea

Strings, listas homogéneas y paralelas, tuplas, funciones, excepciones y modularización

La aerolínea AeroSur necesita procesar la información del equipaje despachado por los pasajeros de un vuelo.

Para representar los datos no se utilizarán listas heterogéneas. En su lugar, se trabajará con listas paralelas: cada lista almacenará un único tipo de dato y la misma posición representará al mismo pasajero.

Se dispone de las siguientes listas homogéneas:

```
codigos = ["AR-104", "AR-205", "AR-316", "AR-421", "AR-522"]
pasajeros = ["Perez Ana", "Gomez Luis", "ROJAS MARTA", "Suarez Juan", "lopez maria"]
categorias = ["E", "P", "E", "B", "E"]
pesos = [18.5, 25.0, 31.0, 20.0, 23.5]
```

Por ejemplo, los datos ubicados en la posición 2 de las cuatro listas corresponden al mismo pasajero: código AR-316, nombre ROJAS MARTA, categoría E y peso 31.0 kg.

Las categorías posibles son E → Economy, P → Premium y B → Business. Los límites de equipaje son: Economy 20 kg, Premium 25 kg y Business 30 kg.

La solución deberá pensarse separando las funciones de procesamiento en un módulo denominado equipaje y el programa que utiliza esas funciones en un programa principal.

a) normalizar_pasajero()

Recibe el nombre y apellido de un pasajero. Debe eliminar espacios iniciales y finales y convertir el texto a formato título.

```
normalizar_pasajero(" ROJAS MARTA ") → "Rojas Marta"
```

b) codigo_valido()

Recibe un código de pasajero. Se considera válido cuando posee exactamente 6 caracteres, comienza con "AR-" y sus últimos tres caracteres son dígitos. La validación deberá ignorar espacios iniciales/finales y aceptar el prefijo escrito en minúscula. Debe devolver True o False.

```
AR-104 → válido
ar-205 → válido
AR104 → inválido
AR-ABC → inválido
BR-125 → inválido
```

c) calcular_exceso()

Recibe peso y categoría. Debe devolver la cantidad de kilogramos excedidos de acuerdo con el límite de la categoría. Si el pasajero no supera el máximo permitido, devuelve 0.

```
calcular_exceso(31, "E") → 11
```

d) registrar_equipaje()

Recibe codigos, pasajeros, categorias y pesos. Debe solicitar un nuevo código, nombre, categoría y peso. El código deberá ser válido y no estar repetido; el nombre no puede quedar vacío; la categoría debe ser E, P o B; y el peso debe ser mayor que 0.

La conversión del peso deberá protegerse mediante try/except. Si todos los datos son válidos, se agregará cada dato a su lista correspondiente y la función devolverá True. Ante cualquier dato inválido, no se modificará ninguna lista y devolverá False.

e) procesar Equipajes()

Recibe códigos, pasajeros, pesos y categorías. Debe recorrer las listas en paralelo, normalizar cada nombre, verificar el código y calcular el exceso. Deberá devolver una lista cuyos elementos sean tuplas del mismo formato:

```
(codigo, nombre_normalizado, exceso)
```

Solamente se incluirán pasajeros cuyo código sea válido. Las listas originales no deberán modificarse.

f) mayor_exceso()

Recibe la lista generada por procesar Equipajes(). Debe devolver una tupla (codigo_pasajero, exceso) correspondiente al pasajero con mayor exceso. Si ningún pasajero posee exceso, deberá devolver ("", 0).

Programa principal

Utilizar las cuatro listas dadas, invocar las funciones del módulo y mostrar únicamente los pasajeros con exceso. Al final, informar el pasajero con mayor exceso.

```
PASAJEROS CON EXCESO
```

```
AR-316 - Rojas Marta - 11.0 kg
```

```
AR-522 - Lopez Maria - 3.5 kg
```

```
Mayor exceso: AR-316 - 11.0 kg
```

EJERCICIO 3 – Plataforma de reservas para un complejo de cines

Ejercicio integrador: matrices, strings, tuplas, excepciones y actualización de datos

El complejo CineCenter posee 4 salas. Durante una función se registran las butacas vendidas mediante una matriz. Cada sala dispone de 8 sectores de venta para simplificar el problema.

```
ventas = [
    [10, 12, 8, 15, 10, 11, 14, 7],
    [8, 0, 12, 14, 6, 9, 11, 5],
    [15, 14, 13, 12, 15, 15, 10, 8],
    [5, 7, 0, 8, 6, 10, 9, 4]
]
salas = ["Sala 1", "Sala 2", "Sala 3", "Sala 4"]
peliculas = ["Mision Lunar", "Codigo Rojo", "El Ultimo Viaje", "Horizonte"]
```

El máximo de entradas permitido por sector es de 15.

Parte 1 – Validación de código de reserva

Cada reserva tiene un código con formato CC-9999. Es decir: exactamente 7 caracteres, comienza con "CC-" y termina con cuatro dígitos.

Desarrollar `validar_reserva(codigo)`, que devuelva `True` o `False`. La función deberá aceptar códigos escritos en minúscula, por ejemplo `cc-1258`.

Parte 2 – Estadísticas por sala

Desarrollar `estadisticas_sala(ventas, salas)`. La función debe devolver una lista de tuplas. Por cada sala se deberá generar:

```
(nombre_sala, total_entradas, sectores_completos)
```

Se considera completo un sector que tenga exactamente 15 entradas vendidas.

Parte 3 – Buscar película

Desarrollar `buscar_pelicula(nombre, peliculas, salas, ventas)`. La búsqueda debe ignorar mayúsculas y minúsculas y eliminar espacios iniciales y finales.

Si la película existe, devolver una tupla (`nombre_pelicula`, `sala`, `entradas_vendidas`). Si no existe, devolver `("", "", 0)`.

Parte 4 – Registrar una nueva venta

Desarrollar `registrar_venta(ventas)`. La función deberá solicitar:

- Número de sala: 1 a 4.
- Número de sector: 1 a 8.
- Cantidad de entradas: número entero positivo.

Todos los ingresos numéricos deberán estar protegidos utilizando manejo de excepciones. No se podrá registrar la operación si:

- la sala no existe;

- el sector no existe;
- la cantidad no es positiva;
- la nueva cantidad provoca que el sector supere las 15 entradas.

Si los datos son correctos, se deberá modificar la matriz. La función devuelve True si pudo realizar la venta y False en caso contrario. No es necesario volver a pedir los datos cuando son inválidos.

Parte 5 – Informe general

Desarrollar `informe_general(ventas, salas, peliculas)`. Debe mostrar:

1. Entradas vendidas por cada sala.
2. Porcentaje que representa cada sala sobre el total vendido.
3. Sala con mayor cantidad de entradas.
4. Películas que poseen al menos un sector completo.

INFORME DE VENTAS

```
Sala 1 - Mision Lunar
Total vendido: 87
Porcentaje: 28.7 %
```

```
...
```

```
Sala con mayor venta: Sala 3
```

PELICULAS CON SECTORES COMPLETOS

```
Mision Lunar
El Ultimo Viaje
```